

Adaptive Auto-Harness: Sustained Self-Improvement for Agentic System Deployment on Open-Ended Task Streams

Zewen Liu¹, Zhan Shi², Yisi Sang², Bing He², Minhua Lin³, Tianxin Wei⁴

Dakuo Wang⁵, Benoit Dumoulin², Wei Jin¹, Hanqing Lu²

¹Emory University ²Amazon ³The Pennsylvania State University ⁴UIUC ⁵Northeastern University
{zewen.liu, wei.jin}@emory.edu; mfl5681@psu.edu; twei10@illinois.edu;
{shizhans, yisisang, binghee, dumoulib, luhanqin}@amazon.com; d.wang@northeastern.edu

Abstract

Auto-harness systems such as A-Evolve, GEPA, and Meta-Harness improve LLM agents by optimizing prompts, skills, tools, memories, and supporting infrastructure from execution feedback, but they are typically evaluated on fixed offline benchmarks. Real deployments instead present open-ended task streams: histories grow without a fixed endpoint, heterogeneous tasks require different harnesses, and problem distributions shift over time. These challenges make a single repeatedly and densely updated harness brittle, causing performance degradation as accuracy peaks early and then declines. This motivates sustained harness construction with task-wise adaptation. We introduce Adaptive Auto-Harness, a framework and system for such streams. The framework decomposes the gap to an oracle harness into evolution loss and adaptation loss. The system addresses these losses with a stateful multi-agent evolver, a harness tree with solve-time routing, and human-steering hooks for cases where history lacks the needed signal. Across prediction-market, security-competition, and event-forecasting streams, Adaptive Auto-Harness outperforms five existing auto-harness baselines and ablations attribute gains to better construction, routing, or targeted human steering.

1 Introduction

Open-ended task streams are a common deployment regime for LLM agents: tasks arrive continuously, feedback accumulates over time, and future tasks may differ from earlier ones. In this regime, an agent’s harness, comprising the prompts, skills, tools, and supporting infrastructure that surround a fixed LLM, is a primary determinant of task-solving performance. *Auto-harness systems* such as A-Evolve (Lin et al., 2026), GEPA (Agrawal et al., 2025), and Meta-Harness (Lee et al., 2026) perform evolution to construct harness automatically

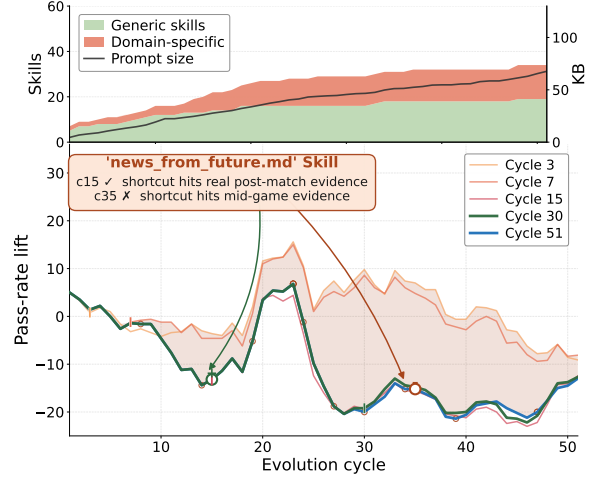


Figure 1: Longer frequent evolution can overfit earlier stream evidence. **Top:** The A-Evolve run with unbounded evolution grows from 12 to 34 skills, while the prompt grows from 2 KB to 68 KB. **Bottom:** Each curve reports pass-rate increase relative to the no-evolution solver, with different evolution stopping cycles. Early gains fade as later tasks arrive; `news_from_future.md` marks an overfitting example of the evolved skill.

from execution feedback, and report substantial gains on static offline benchmarks such as SWE-bench (Jimenez et al., 2024). Those evaluations, however, do not capture the central pressure of deployment: the harness must keep improving while operating on a chronological stream whose history grows, task types vary, and distribution shifts.

Representative streams include prediction markets with thousands of questions over weeks (Cheng et al., 2026), decade-long security competitions (Zhuo et al., 2025), and cross-lingual forecasting services with heterogeneous sources (Zeng et al., 2025). Figure 1 illustrates why repeatedly evolve and inject all harness during solving is insufficient. On Polymarket: we run A-Evolve and stop evolution after 3, 7, 15, 30, or 51 cycles, comparing each run with the same solver without evolution. Early evolution improves pass rate, but longer runs accumulate larger

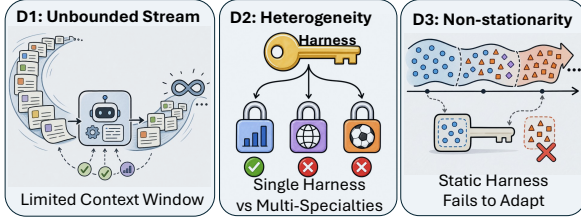


Figure 2: **Three deployment dimensions in open-ended task streams.** Unbounded stream, heterogeneous tasks, and non-stationary distributions expose the limits of evolving a single dense harness for long-term deployment

prompts and more specialized skills. For example, `news_from_future.md` first detects post-match evidence and later fires on mid-game evidence, turning a useful temporal cue into a stale shortcut. All stopping budgets eventually peak and decline; later in the stream, shorter runs outperform longer ones. Sustained deployment therefore requires preserving useful history while adapting the active harness to the task at hand.

This failure exposes three deployment dimensions that static benchmark evaluation does not capture (Figure 2). **(D1) Unbounded Streams.** The task stream has no fixed train/test cutoff or designated endpoint (Karten et al., 2026; Wang et al., 2023); feedback, trajectories, and harness state accumulate throughout deployment, creating a heavy burden for the evolution. Existing auto-harness systems built around a single-agent evolver compress this expanding history into a finite context window, creating a bottleneck for building the effective and generalizable harness. **(D2) Task heterogeneity.** Varied types of tasks are mixed in the same stream. A prediction-market platform, for example, mixes politics, sports, and finance questions in the same hour, each calling for distinct sources, tools, and prompting. However, existing auto-harness systems deploy a static dense harness across the stream, with no solve-time adaptation to the task at hand, and neglects the fact that a single fixed policy is rarely optimal across heterogeneous problems (Miao et al., 2025). **(D3) Distributional non-stationarity.** As the stream progresses, incoming tasks shift away from the experience the harness was last fitted on. A harness optimized for recent cycles therefore drifts out of fit for new tasks, even with rich historical experience. Closing this gap requires per-task contextual adaptation of the harness, not only continued historical fitting. Additional diagnostics are shown in Appendix A.

We address these dimensions with a unified analytical framework and identify two root gaps that prior auto-harness systems overlook. This analysis motivates: (1) *Sustained auto-harness* (§3.3), which replaces stateless one-shot evolution with a stateful multi-agent system with cross-cycle knowledge for better harness construction; and (2) *Solve-time adaptation* (§3.4), which adapts a task-relevant harness for each problem prior to solving, restoring per-task fit on a heterogeneous, drifting stream. Beyond the two axes, we further introduce a third axis: human-in-the-loop (HITL) for auxiliary steering of the harness to incorporate human insights and provisions that are absent from historical experience. Our contributions are:

- (1) **Deployment-regime analysis of open-ended agentic streams.** We formalize why static auto-harness evaluation is insufficient once tasks arrive as unbounded, heterogeneous, and non-stationary streams. The framework decomposes the gap towards the optimal harness into evolution loss and adaptation loss, providing guidance for auto-harness system designs.
- (2) **Adaptive Auto-harness system.** We introduce a stateful multi-agent evolver for sustained harness construction, a harness-tree router for solve-time adaptation, and structurally triggered human-in-the-loop hooks for evolutions where historical experience is insufficient.
- (3) **Comprehensive empirical validation and diagnosis.** We evaluate on three streaming tasks spanning prediction markets, security challenges, and event forecasting against other auto-harness systems. Beyond aggregated performance, we provide in-depth analysis and evidence for the proposed gaps, component ablations, and human-in-the-loop slice analyses.

2 Related Work

Continual Learning in Task Streams. Continual learning studies systems that learn from a sequence of tasks while retaining earlier capabilities (Buzzega et al., 2020; Wang et al., 2022b,a). Domain and test-time adaptation address distribution shift (Ben-David et al., 2010; Ganin et al., 2016; Wang et al., 2021; Liang et al., 2020), and mixture-of-experts methods route heterogeneous inputs to specialised components (Jacobs et al., 1991; Shazeer et al., 2017). These directions cover pieces of our D1–D3 setting, but they usually adapt model weights, classifiers, or expert modules. In

contrast, we study the harness-level analogue.

Self-improving and self-evolving agents. The closest LLM-agent precedents to our setting are systems that update the harness directly from execution feedback. A-Evolve (Lin et al., 2026) introduces a linear-chain evolver: each cycle reads batch trajectories and mutates prompts, skills, memory, and tools. GEPA (Agrawal et al., 2025) adds reflective Pareto prompt evolution using textual feedback rather than scalar reward. Meta-Harness (Lee et al., 2026) uses a growing filesystem archive and Claude Code as the proposer. Continual Harness (Karten et al., 2026) enables online adaptation within a single continuous deployment run through alternating action/refinement cycles. SkillOS (Ouyang et al., 2026) learns a skill curation policy via reinforcement learning, training a curator to select and refine reusable skills from repeated interactions.

3 Method

3.1 Problem Formulation

We consider tasks arriving in an open-ended stream $x_1, x_2, \dots, x_T$ with $x_t \sim P_t$, where P_t is the task distribution at time t and each x_t has a fixed ground truth $y(x_t)$. The agent observes the history of prior experience $\mathcal{H}_t = \{(x_i, r_i, \tau_i)\}_{i=1}^{t-1}$, where r_i is optionally the realized reward on task x_i and τ_i is the agent’s solving trajectory (actions, intermediate observations, tool calls). A *harness* $C = \varphi(\mathcal{H}_t)$, with $|C| \leq K$, is a bounded representation (prompts, skills, memory, and tools) with capacity budget K , where φ is the *evolver*: the agentic system that automatically transforms historic experience into the harness. A solver agent then acts via the policy $a_t \sim \pi(a \mid x_t, C_t)$, where π is the LLM-induced sampling distribution over actions conditioned on the task and harness.

3.2 Analytical Framework

The three deployment dimensions of section §1 surface concrete failures, but they do not by themselves identify what an evolver should fix. To pinpoint the root causes, we frame the problem analytically: we cast harness construction as a regret-minimization problem against an oracle reference, and decompose the regret into two complementary loss terms that map directly to actionable design axes.

Utility and regret. We define the utility of a harness C on a task x_t as

$$V(C, x_t) = \mathbb{E}_{a_t \sim \pi(\cdot \mid x_t, C)}[r(a_t, y(x_t))], \quad (1)$$

which is the expected reward of the harness-conditioned solver. The full-history utility is the corresponding ceiling under the capacity budget, $V(\mathcal{H}_t, x_t) := \sup_{C: |C| \leq K} V(C, x_t)$, that is, the best utility attainable by any bounded harness on x_t . The regret of the evolver-constructed harness is then

$$\text{Regret}(\varphi, x_t) = V(\mathcal{H}_t, x_t) - V(\varphi(\mathcal{H}_t), x_t) \geq 0, \quad (2)$$

non-negative since $\varphi(\mathcal{H}_t)$ is one element of the supremum’s domain. Operationally, this ceiling matches what a solver granted $\mathcal{H}_t$ directly, under the same compute and tool budget as φ , could attain by reconstructing any candidate harness on the fly. **Solve-time optimal harness.** Fix the evolver class Φ and define the solve-time optimal harness for a particular task:

$$C_\Phi^*(x_t) = \arg \max_{\substack{C = \varphi(\mathcal{H}_t, x_t), \varphi \in \Phi \\ |C| \leq K}} V(C, x_t). \quad (3)$$

This is an hypothetical oracle reference: it is the best harness the evolver class Φ could produce if it were allowed to condition on the incoming task x_t at solve time. A deployed evolver φ commits to one harness $\varphi(\mathcal{H}_t)$ before x_t is observed; using C_Φ^* as pivot between $V(\mathcal{H}_t, x_t)$ and $V(\varphi(\mathcal{H}_t), x_t)$ yields the following decomposition.

Proposition 1 (Regret Decomposition). *For any deployed evolver $\varphi \in \Phi$,*

$$\mathbb{E}_{x_t}[\text{Regret}(\varphi, x_t)] = L_{\text{evo}}(\Phi) + L_{\text{adapt}}(\varphi), \quad (4)$$

where

$$L_{\text{evo}}(\Phi) = \mathbb{E}_{x_t}[V(\mathcal{H}_t, x_t) - V(C_\Phi^*(x_t), x_t)], \quad (5)$$

$$L_{\text{adapt}}(\varphi) = \mathbb{E}_{x_t}[V(C_\Phi^*(x_t), x_t) - V(\varphi(\mathcal{H}_t), x_t)]. \quad (6)$$

L_{evo} is the *evolution loss*: it reflects the evolver class Φ ’s capability gap, regardless of how many cycles the evolver runs. A single-agent prompt editor cannot produce multi-file infrastructure; that ceiling is structural, not a matter of effort. Reducing L_{evo} therefore requires pursuing more capable evolver systems with access to broader control and feedbacks. L_{adapt} is the *adaptation loss*: the oracle builds the optimal harness per task, but a deployed φ commits to one harness before seeing x_t . Therefore, even with an optimal evolver system, L_{adapt} exists as long as task heterogeneity persists.

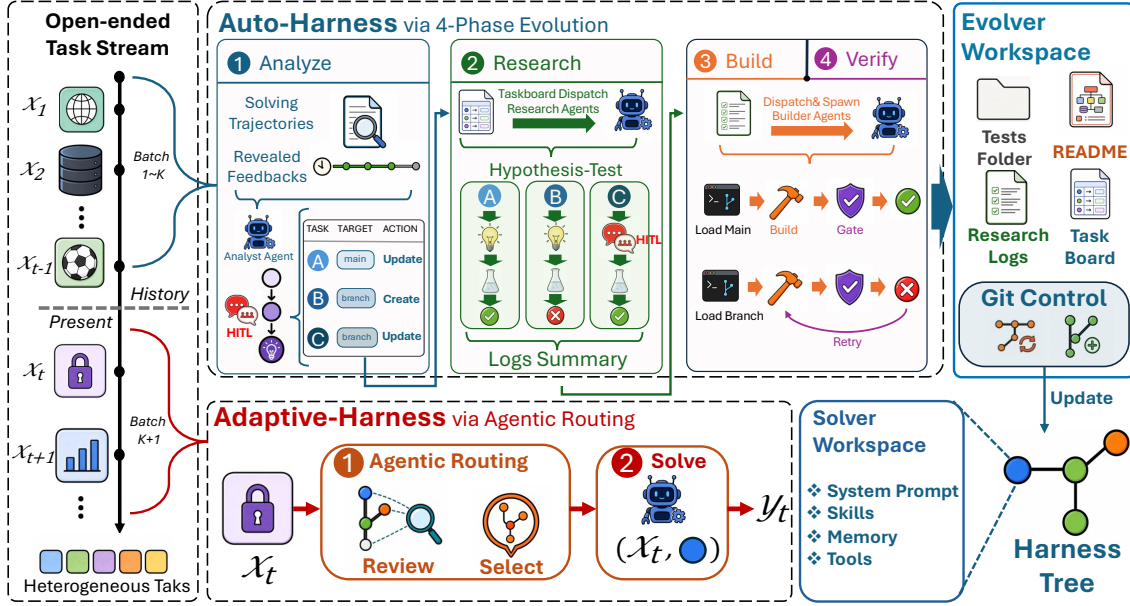


Figure 3: Overview of the Adaptive Auto-Harness system. **Top:** the multi-agent evolver constructs and refines a harness tree across cycles via four phases (Analyze $\rightarrow$ Research $\rightarrow$ Build $\rightarrow$ Verify) with a persistent cross-cycle workspace and temporal-reveal feedback. **Bottom:** at solve time, a router agent reads each branch’s workspace via git show and routes the incoming task x_t to the most suitable branch. Two human-in-the-loop hooks (task-board steering and research-phase assistance) trigger only when the evolver’s history lacks relevant signal.

Human-in-the-loop as a third axis. The decomposition assumes $\mathcal{H}_t$ contains relevant signal for x_t . When it does not, the regret decomposition no longer applies; we address this case via a third axis outside $L_{\text{evo}} + L_{\text{adapt}}$, the human-in-the-loop channel (§3.5).

3.3 Sustained Auto-Harness via Multi-Agent Evolution

The analytical decomposition identifies L_{evo} as the loss from harness capabilities that the evolver class cannot construct from history. We reduce this loss by expanding Φ with a stateful four-phase multi-agent evolver, temporal-reveal feedback, and cross-cycle memory (Figure 3). This design targets the unbounded-stream failure where a single-agent evolver must absorb growing trajectories, delayed labels, and prior research within one context window. Concretely, we address three structural limitations:

(1) Multi-agent with distinct roles and objectives. Because unbounded streams keep expanding the trajectory history an evolver must interpret, existing single-agent evolvers must fit analysis, research, implementation, and verification into one context window. We decompose evolution into four phases (Analyze $\rightarrow$ Research $\rightarrow$ Build $\rightarrow$ Verify), each with a dedicated objective and full context budget.

This eliminates the single-window bottleneck and lets parallel Researchers explore disjoint hypotheses without premature convergence.

(2) Temporal-reveal feedback. Under unbounded streams, labels arrive asynchronously (e.g., a prediction market resolves days after the trade). We implement a *temporal-reveal gate* that surfaces each task’s evaluation signal to the evolver only after its resolution date, providing a proper streaming feedback signal without leaking future information.

(3) Persistent cross-cycle state. We provide the evolver with a dedicated workspace that persists across cycles, containing a task board (prioritised failure analysis), research logs (tested hypotheses with pass/fail verdicts), architecture documentation (README), and verification tests. This cross-cycle memory enables the evolver to refine its construction ability over time and build upon prior evolution experience rather than restarting from scratch.

3.4 Solve-Time Adaptation via Harness-Tree Routing

The analytical decomposition identifies L_{adapt} as the loss from committing to one dense harness before observing the incoming task’s context. We reduce this loss by shifting heavy adaptation into evolution time: the evolver constructs a struc-

tured harness store, and solve time only requires a lightweight adaptation operator. The reduction is general: it factors into (i) how the evolver organizes the harness space and (ii) how the solver selects from that space per task. The harness space can be organized in many forms, including a linear chain that always uses the most recently evolved workspace, a tree of regime-specific branches, or a graph of skills with dependency edges. The adaptation operator can likewise take many forms, ranging from skill-level retrieval over a flat catalog (ByteDance, 2025) to branch-level routing over a structured space. Different combinations trade off construction cost, operator latency, and the granularity at which adaptation occurs; how to construct the space is the job of evolution and is outside the scope of this section.

We adopt a *harness tree* as the storage form and *agentic routing* as the adaptation operator. The tree is the natural fit for our setting because heterogeneous task streams cluster into a small number of recurring regimes (e.g., binary exploitation versus cryptography in CTF-Dojo, sports versus politics in PolyBench), branches isolate regime-specific prompts/skills/tools without cross-contaminating the others, and the branching gate gives the evolver an explicit lever to commit specialization only when warranted by failure evidence. We instantiate this with two designs (Figure 3). **(1) Branching harness tree (built at evolution time).** The solver’s workspace is a git repository. The evolver constructs regime-specific branches (e.g., branch/crypto-classical, branch/binary-reversing) during evolution, each carrying its own prompt, skills, and tool registry; git provides versioning, isolation, and lineage tracking across branches. **(2) Agentic routing (executed at solve time).** A router agent reads each branch’s workspace via `git show` and selects the branch given x_t ’s context; the solver then checks out that branch and executes.

3.5 Human-in-the-Loop Channel

A third failure lies outside $L_{\text{evo}} + L_{\text{adapt}}$: some tasks require harness absent from $\mathcal{H}_t$ signal, such as API credentials, novel web sources, or proprietary endpoints. In this experience-insufficient setting, neither a stronger evolver nor solve-time routing can recover the missing signal. We address it with a human-in-the-loop channel that augments $\mathcal{H}_t$ through structurally triggered steering hooks (Figure 3). This design targets open-ended

Table 1: Benchmark statistics for the three chronological task streams.

Bench.	Tasks	Span	Resolution	Domain
PolyBench	5,075	Feb 6–22, 2026	Day	Prediction markets
CTF-Dojo	261	2011–2024	Year	Security
FutureX	503	Jan–Apr 2026	Week	Forecasting

streams where new access requirements appear before autonomous evolution has relevant evidence.

(1) Task-board steering. After the Analyst updates the task board, a human may review it to add entries, adjust priorities, or supply domain guidance and source access. This proactively steers the subsequent research cycle with direction the evolver cannot derive from trajectories alone. **(2) Interactive assistance during research.** When a Researcher agent hits a barrier mid-execution that requires human intervention (e.g., an authentication wall), the hook prompts the human in real time. This reactively unblocks the research agent at the point of failure.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on three open-ended task streams (Table 1): *PolyBench* for prediction markets (Cheng et al., 2026), *CTF-Dojo* for security challenges (Zhuo et al., 2025), and *FutureX* for event forecasting (Zeng et al., 2025). All three enforce strict temporal order and covers the three dimensions of challenge. Details and non-stationarity diagnostics are in Appendix A and C.

Baselines. We compare with no-evolution runs with different solver agents using **Sonnet-4.6** (Anthropic, 2025b), **DeepSeek-V3.2** (DeepSeek-AI, 2025), **Claude Haiku** (Anthropic, 2025a), **textbfGLM-4.7** (Z.ai, 2025), and **Kimi-K2.5** (Moonshot AI, 2026). We compare with five auto-harness baselines: **A-Evolve** (Lin et al., 2026), **GEPA** (Agrawal et al., 2025), **Meta-Harness** (Lee et al., 2026), **Continual Harness**, and **Skillos**. We also compare with one human-designed system **OctoTools** (Lu et al., 2025).

Solver and evolver. We use Claude Sonnet 4.6 as the solver for all experiments unless specified, and use Claude Opus 4.6 as the evolver, both at temperature 0 to attribute gains to the evolution algorithm rather than sampling noise; the no-evolution controls additionally report base-agent results for Haiku 4.5, DeepSeek-V3.2, Kimi-K2.5, and GLM-4.7. All algorithms share the

Table 2: Main comparison across no-evolution agents, auto-harness baselines, a human-designed system, and our three variants. PolyBench reports Coverage / Accuracy / Return (coverage-scaled CWR, in %); CTF-Dojo and FutureX report the official Pass@1. Bold marks the best result per row and underline marks second best.

Benchmark	Metric	No evolution					Auto-harness baselines					Human	Ours		
		Sonnet	Haiku	DeepSeek	Kimi	GLM	A-Evolve	GEPA	Meta Harness	Cont. Harness	SkillIOS	OctoTools	Multi agent	Adaptive	Full System
PolyBench	Accuracy $\uparrow$	22.2	15.2	1.4	14.0	14.0	18.4	13.4	50.8	8.5	21.4	40.0	<u>79.8</u>	77.4	80.9
	Return $\uparrow$	+1.7	+88.0	+16.4	-2.0	+10.1	+7.2	+0.2	+320	+1.7	+3.6	+20.4	<u>+351</u>	+352	+330
CTF-Dojo	Pass $\uparrow$	37.2	23.8	26.1	24.5	12.6	45.2	42.9	41.0	25.7	29.5	38.3	<u>47.9</u>	46.0	50.2
FutureX	Pass $\uparrow$	31.0	31.0	31.2	27.8	30.8	<u>47.5</u>	28.2	29.4	31.8	29.8	25.6	49.5	44.1	47.3

same batch size (100/20/20 for PolyBench/CTF-Dojo/FutureX), batch loop, and temporal-reveal gate; only the evolution algorithm varies.

Metrics. For PolyBench, we report three complementary metrics: *Accuracy*, the fraction of all markets traded correctly; and *Return*, defined as Coverage $\times$ CWR, where CWR is the confidence-weighted portfolio profit-to-investment ratio over traded markets and coverage is the fraction of traded. For CTF-Dojo and FutureX, we report the official pass rate (Pass@1) defined by the original benchmarks (Zhuo et al., 2025; Zeng et al., 2025). We also report *lift* in the figures as the difference with baselines. Full definitions are in Appendix A.

Research questions. **RQ1** (§4.2): How does Adaptive Auto-Harness compare against existing auto-harness systems on open-ended task streams? **RQ2** (§4.3): How does Adaptive Auto-Harness address benchmark-specific bottlenecks? **RQ3** (§4.4): Does stateful multi-agent evolution with evaluation feedback provide additional gains? **RQ4** (§4.5): Can solve-time routing effectively leverage specialized harness branches? **RQ5** (§4.6): Can human steering help under insufficient history signal?

4.2 Comparison with Baselines

Prior systems specialize on one metric cluster. A-Evolve leads the two pass-rate streams (45.2% CTF-Dojo, 47.5% FutureX) but covers only 21.1% of PolyBench markets. Meta-Harness leads all three PolyBench metrics (55.3% Coverage, 50.8% Accuracy, +320% Return) but falls below the no-evolution Sonnet baseline on FutureX (29.4% vs. 31.0%). Base solvers stay below 32.6% PolyBench Coverage on every model, and OctoTools, the frozen human-designed system, places third on PolyBench Return but does not lead any row. The pass-rate cluster and the portfolio cluster therefore sit in different baselines.

Our three variants jointly lead all five metrics. Without HITL intervention, the Full System combines multi-agent evolution with solve-time routing

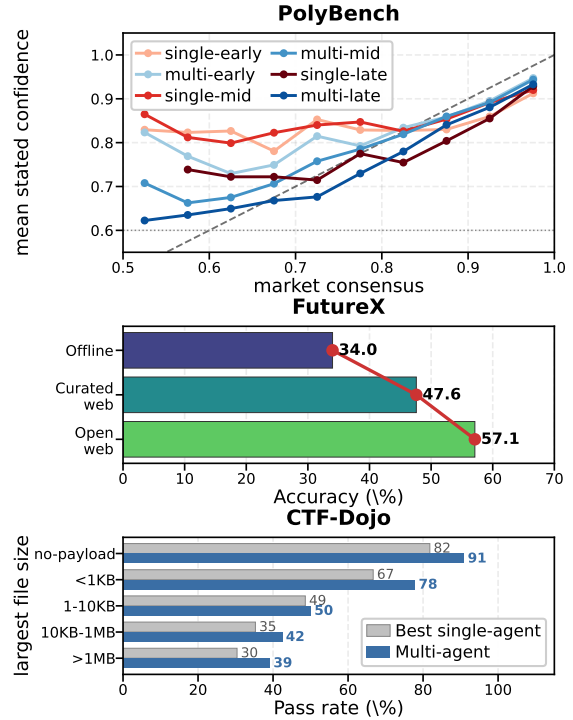


Figure 4: L_{evo} evidence across benchmark-specific bottlenecks. PolyBench stresses confidence calibration, FutureX stresses web-retrieval access, and CTF-Dojo stresses payload handling of different file sizes.

and reaches 97.9% PolyBench Coverage, 80.9% Accuracy, and 50.2% CTF-Dojo Pass. The Multi-agent variant leads FutureX at 49.5%, where evolving the right source and tooling matters more than per-task routing. The Adaptive variant leads PolyBench Return at +352%, where matching each market to a specialized strategy matters.

4.3 Benchmark Bottlenecks

RQ2 asks whether Adaptive Auto-Harness targets the bottlenecks that limit each benchmark. We organize the analysis around the two losses introduced in §3.2: the *evolution loss* L_{evo} , the gap from capabilities the evolver class cannot construct; and the *adaptation loss* L_{adapt} , the gap from committing to a single harness across heterogeneous tasks.

Evolution bottlenecks (L_{evo}) differ across bench-

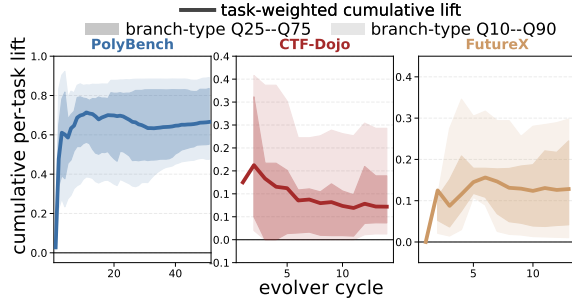


Figure 5: L_{adapt} evidence from branch specialization. Curves show cumulative routing lift over the baselines, while shaded bands show harness branch performance spread across cycles.

marks. Figure 4 plots a benchmark-specific stress axis chosen to expose the most predictive capability. The PolyBench panel plots mean stated confidence against market consensus, defined as the implied probability of the favored outcome from Polymarket prices; a well-calibrated harness from our multi-agent variant tracks the diagonal, while single-agent variants stay flat and over-state confidence on low-consensus markets, suggesting that the binding capability is consensus-aware confidence calibration rather than raw prediction skill. The FutureX panel plots pass rate against three retrieval tiers, ranging from offline to date-filtered Wikipedia plus DuckDuckGo to unrestricted DuckDuckGo web search. Pass rate increases monotonically from 34.0% to 47.6% to 57.1%, identifying source acquisition rather than reasoning as the binding capability. The CTF-Dojo panel plots pass rate against the largest challenge file size, binned into five tiers from no-payload to >1MB. The best single-agent variant declines from 81.8% to 30.4%, and the multi-agent variant declines from 90.9% to 39.1% while retaining roughly a 9-point margin throughout, indicating that payload-handling infrastructure becomes the binding capability as inputs grow and that the multi-agent evolver mitigates but does not eliminate this bottleneck. In summary, the binding capability therefore differs by benchmark.

Adaptation bottlenecks (L_{adapt}) remain after evolution. Figure 5 reports per-task routing lift over evolver cycles, where lift on each task is the routed-harness score minus the median score across single-committed-harness baselines. The solid curve is the within-cycle mean lift and the shaded band is performance variation across different branches; if a single committed dense harness were sufficient, the mean lift would approach zero as evolution progresses. We observe instead that

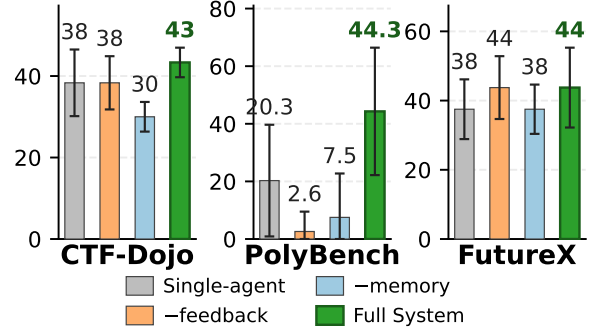


Figure 6: Ablations for multi-agent evolution. Removing evaluation feedback or cross-cycle memory degrades performance relative to the full stateful system.

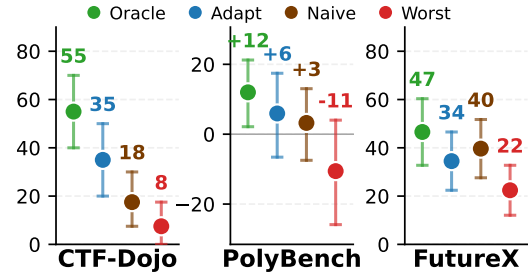


Figure 7: Solve-time routing evaluation on an evolved harness tree. Oracle reports the best branch performance per task, Adapt uses our agentic router, Naive always uses main workspace, and Worst reports the worst branch performance per task.

the mean lift and variation remains positive across all cycles on all three benchmarks.

Takeaway. Although streams expose different bottlenecks, they can be addressed under the same principle: reduce evolution loss for missing capabilities and adaptation loss for task-specific fit.

4.4 Stateful Multi-Agent Evolution

RQ3 asks whether the four-role evolver in section §3.3 (Analyst → parallel Researchers → Builder → Verifier) improves over a single-agent evolver, and whether its state channels matter. Using a subset of samples, figure 6 compares the full system with variants that remove temporal-reveal feedback or cross-cycle memory. The full system improves over the single-agent evolver on all three benchmarks: 20.3 → 44.3 CWR on PolyBench, 38% → 43% on CTF-Dojo, and 38% → 44% on FutureX. Removing memory causes the broadest degradation, while removing feedback mainly hurts PolyBench, where outcomes resolve after trading.

Takeaway. The four-role evolver is strongest when paired with persistent state: memory preserves cross-cycle search, and feedback turns resolved outcomes into later evolution signal.

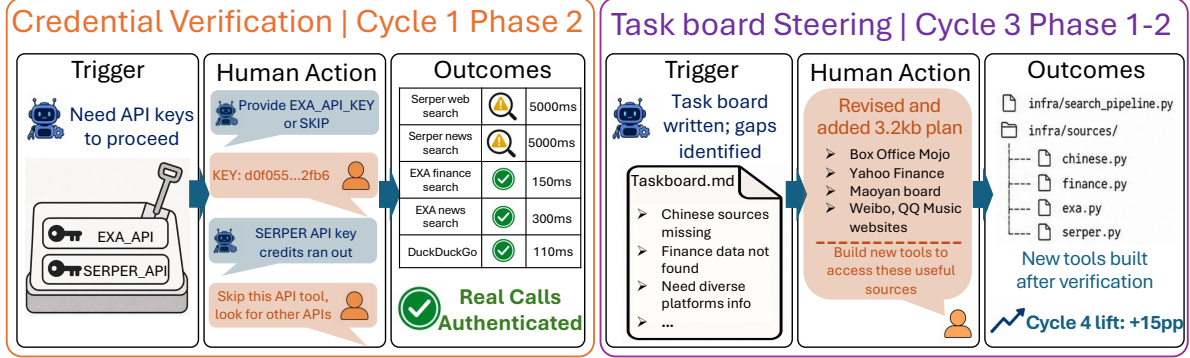
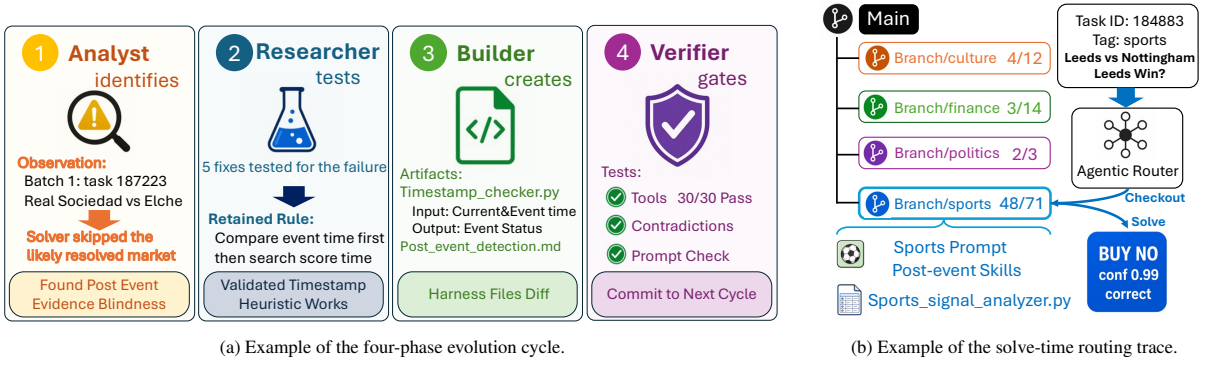


Figure 8: Extracted trajectories of the designed multi-agent evolution, agentic routing, and Human-in-the-Loop.

4.5 Solve-Time Routing on the Harness Tree

RQ4 asks whether solve-time routing can leverage the specialized harness tree. We run our algorithm on a subset of samples and then replay all branches on the same tasks and compare *Oracle* (best branch per task), *Adapt* (our router), *Naive* (main), and *Worst* (worst branch per task). Figure 7 shows clear Oracle-to-Naive headroom: 37 points on CTF-Dojo, +9 CWR on PolyBench, and 7 points on FutureX. Adapt captures the gap on CTF-Dojo and PolyBench, but trails Naive on FutureX, where source acquisition dominates routing choice.

Takeaway. Harness specialization creates headroom across streams, but FutureX shows that routing cannot compensate when source acquisition dominates branch choice.

4.6 Human Steering for Auto-Harnessing

RQ5 asks whether human steering helps when history lacks the source or access signal needed for evolution. Since all previous experiment does not apply HITL to ensure fairness, we evaluate this on FutureX and restrict human input to two hooks: *research-phase steering* supplies credentials when research is blocked, and *task-board steering* adds source directions the evolver cannot infer (Figure 8c). Figure 9 shows the slice-level effect: lift

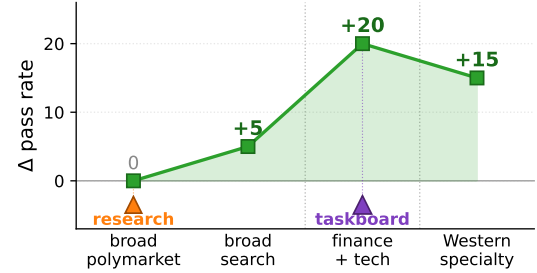


Figure 9: FutureX pass-rate lift over four task slices under two human-steering hooks. The orange triangle marks research-phase steering and the purple triangle marks task-board steering. Lift is measured against the no-HITL run.

is 0 on broad polymarket questions, rises to +5 on broad search-dependent questions, peaks at +20 on the directly targeted finance&tech slice, and remains +15 on adjacent Western-specialty questions. The pattern indicates that HITL helps when it injects the missing external signal, not generic human advice.

Takeaway. Human steering is most useful when the missing ingredient is external source knowledge rather than additional autonomous evolution.

5 Conclusions

Open-ended task streams expose three challenges for auto-harness deployment: unbounded task ar-

rival, heterogeneous tasks, and non-stationarity. Adaptive Auto-Harness addresses these challenges by pairing sustained harness construction with solve-time task adaptation. The decomposition into evolution loss and adaptation loss clarifies why a single repeatedly updated harness is insufficient: the system must build missing capabilities from stream evidence while selecting the right specialized branch for each task. The experiments further show that the three mechanisms are complementary rather than interchangeable: multi-agent evolution constructs benchmark-specific capabilities, routing exploits harness specialization when branch signals are reliable, and human steering supplies external signals that history cannot contain.

6 Limitations

Benchmark coverage. We evaluate on three open-ended task streams: prediction markets, cybersecurity challenges, and event forecasting. These domains cover unbounded streams, task heterogeneity, and distributional non-stationarity, but the same framing should be tested further on additional deployment streams where the stream is further expanded spatially and temporally to mimic the real-world deployment.

Diagnostic losses. The evolution loss L_{evo} and adaptation loss L_{adapt} are analytical quantities, not directly estimated oracle losses. Our experiments diagnose them through bottleneck analyses, ablations, and routing controls rather than through a formal estimator of the oracle harness.

7 Ethics Statement

We use public research benchmark tasks and do not introduce private user data. CTF-Dojo runs only inside isolated benchmark containers and does not target real systems. Human steering is limited to source guidance, task-board edits, and credential decisions; humans do not label/expose answers or choose solver branches.

8 AI Usage Statement

We used AI assistants to refine the writing of this paper and to accelerate debugging and analysis during implementation and evaluation.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan,

Meng Jiang, and 1 others. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.

Anthropic. 2025a. Claude haiku 4.5. <https://www.anthropic.com/claude/haiku>.

Anthropic. 2025b. Claude sonnet 4.6. <https://www.anthropic.com/claude/sonnet>.

Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. 2010. A theory of learning from different domains. *Machine Learning*, 79:151–175.

Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. 2020. Dark experience for general continual learning: A strong, simple baseline. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 15920–15930.

ByteDance. 2025. Deerflow: A skill-based agent framework with dynamic skill loading. <https://github.com/bytedance/deer-flow>. Open-source repository.

Pu Cheng, Juncheng Liu, and Yunshen Long. 2026. Polybench: Benchmarking llm forecasting and trading capabilities on live prediction market data. *arXiv preprint arXiv:2604.14199*.

DeepSeek-AI. 2025. Deepseek-v3.2 technical report. *arXiv preprint arXiv:2512.02556*.

Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2016. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.

Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vardhalli. 2026. Continual harness: Online adaptation for self-improving foundation agents. *arXiv preprint arXiv:2605.09998*.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.

- Jian Liang, Dapeng Hu, and Jiashi Feng. 2020. Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation. In *International Conference on Machine Learning (ICML)*.
- Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, and 1 others. 2026. Position: Agentic evolution is the path to evolving llms. *arXiv preprint arXiv:2602.00359*.
- Pan Lu, Bowen Chen, Sheng Liu, Rahul Thapa, Joseph Boen, and James Zou. 2025. Octotools: An agentic framework with extensible tools for complex reasoning. *arXiv preprint arXiv:2502.11271*.
- Rui Miao, Babak Shahbaba, and Annie Qu. 2025. Reinforcement learning for individual optimal policy from heterogeneous data. *Annals of statistics*, 53(4):1513.
- Moonshot AI. 2026. Kimi k2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chunliang Li, Yizhu Jiao, Kaiwen Zha, and 1 others. 2026. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*.
- Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. 2021. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. 2022a. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *European Conference on Computer Vision (ECCV)*, pages 631–648.
- Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. 2022b. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 139–149.
- Z.ai. 2025. Glm-4.7. <https://www.z.ai/>.
- Zhiyuan Zeng, Jiashuo Liu, Siyuan Chen, Tianci He, Yali Liao, Yixiao Tian, Jinpeng Wang, Zaiyuan Wang, Yang Yang, Lingyue Yin, and 1 others. 2025. Futurex: An advanced live benchmark for llm agents in future prediction. *arXiv preprint arXiv:2508.11987*.
- Terry Yue Zhuo, Dingmin Wang, Hantian Ding, Varun Kumar, and Zijian Wang. 2025. Training language model agents to find vulnerabilities with ctf-doj. *arXiv preprint arXiv:2508.18370*.

A Benchmark and Evaluation Details

Across all three benchmarks, tasks are evaluated in chronological order. The solver receives task x_i with only the information available before its release time, while the evolver receives outcome labels only after the corresponding resolution time. Our analysis pipeline loads the first record for each instance_id, so retries or duplicate logs do not change the reported metrics. The three benchmarks together exercise the three open-ended-stream deployment dimensions identified in §1.

A.1 PolyBench

Source and composition. PolyBench is a Polymarket-derived prediction-market stream with 5,075 tasks from Feb 6–22, 2026. The stream spans politics, sports, finance, crypto, and entertainment markets, and each task is resolved against the official market outcome.

Temporal ordering. For each market, we store the task release timestamp and the outcome resolution timestamp. Resolved labels are hidden from evolution until the corresponding market has resolved.

Metrics. Let N be the total number of markets in the stream and let $\mathcal{T}$ be the set of executed trades, excluding gated tasks, empty decisions, and SKIP decisions. Let $s_i \in \{0, 1\}$ indicate whether trade i is correct, b_i be its confidence-weighted investment, and g_i be its realized profit. The metrics reported for PolyBench are:

$$\begin{aligned}\text{Coverage} &= \frac{|\mathcal{T}|}{N}, \\ \text{Acc} &= 100 \cdot \frac{1}{N} \sum_{i \in \mathcal{T}} s_i, \\ \text{CWR} &= 100 \cdot \frac{\sum_{i \in \mathcal{T}} g_i}{\sum_{i \in \mathcal{T}} b_i}, \\ \text{Return} &= \text{Coverage} \cdot \text{CWR}.\end{aligned}$$

Accuracy therefore rewards both broad coverage and correct decisions. Return is a portfolio-style profitability metric: CWR captures the dollar-weighted profit per unit invested over the markets the agent actually traded, and the Coverage scaling discounts a high CWR earned on only a thin slice of the stream. We report Return alongside Accuracy because Accuracy treats every trade equally and is blind to stake sizing, whereas under confidence-weighted investments a confident wrong trade can offset several confident correct ones; Return therefore reflects realized P&L rather than mere directional correctness.

A.2 CTF-Dojo

Source and composition. CTF-Dojo is a 261-challenge security-competition stream drawn from pwncollege/ctf-archive, chronologically ordered from 2011 to 2024. Challenges cover binary exploitation, web security, cryptography, reverse engineering, and forensics, exposing changes in challenge style and tooling over time.

Sandbox and verification. Each challenge runs inside a per-task Docker sandbox with constrained network policy. Flags are submitted as text and verified by SHA-256 hash comparison against the official flag.

Metrics. We report Pass@1, the percentage of challenges solved within the benchmark budget. All CTF-Dojo aggregate results use the same chronological ordering as the released stream.

A.3 FutureX

Source and composition. FutureX is a 503-question event-forecasting stream over 82 days from Jan–Apr 2026, drawn from FutureX-Past. Questions cover finance, technology, geopolitics, and entertainment, with both English and Chinese-language variants. The zh-finance slice requires source discovery beyond default English-only retrieval.

Temporal retrieval and filtering. FutureX tasks are historical, but web pages and search indices continue to change after the event. To avoid label leakage, each task is solved with a per-task cutoff date derived from its temporal metadata. In strict built-in retrieval, Wikipedia content is fetched through the revision API using the latest revision before the cutoff, DuckDuckGo results are fetched and filtered by extracted publication dates from htmldate and URL patterns, and structured economic series are queried with observation and real-time endpoints capped at the cutoff. For evolved tools executed through the sandbox, live command output is passed through an LLM temporal filter before the solver sees it: the filter receives the task, cutoff date, and retrieved content, then either returns CLEAN or replaces post-cutoff values, rows, and snippets with [REDACTED]. This preserves pre-cutoff evidence while blocking dynamic web content that would reveal the resolved answer.

Metrics. We report Pass@1 under the official FutureX criterion. Slice-level human-steering analyses use the same pass criterion and aggregate tasks by the groups shown in Figure 9.

B Implementation and Reproducibility Details

Execution protocol. All main runs use provider-hosted LLM APIs with native tool calling and the same chronological task order used by the benchmarks. Reported numbers are generated from the corresponding `results.jsonl` files. Closed provider-hosted models do not expose parameter counts; we therefore report the evaluated task counts and evolution cycles rather than GPU-hours. The full-system runs contain 5,075/261/503 solve trajectories and 51/14/26 evolution cycles for PolyBench/CTF-Dojo/FutureX, respectively.

Compute and resources. All model inference is performed through provider-hosted APIs; the primary Adaptive Auto-Harness runs use Claude Sonnet 4.6 for solving and Claude Opus 4.6 for evolution, with other provider-hosted models used only for the corresponding baseline rows. We do not train or fine-tune model weights, and no local GPU compute is used for model optimization. Local compute is used for orchestration, result aggregation, figure generation, and Docker-based benchmark execution, including the per-task CTF-Dojo and FutureX sandboxes. Because the evaluated closed models do not disclose parameter counts, task counts and evolution cycles are the main compute descriptors.

Hyperparameters. Table 3 lists the per-benchmark hyperparameters used by all main runs. Because temperature is zero throughout, the reported metrics are point estimates rather than averages over re-samples.

Seed harness. Table 4 reports what the evolver inherits before any cycle runs. The seed prompt is intentionally compact: PolyBench and CTF-Dojo seed prompts are 27 and 24 lines respectively, FutureX is longer because it documents the temporal-retrieval contract. All three benchmarks start with zero seed skills, tools, and memory entries, so any harness component beyond the seed prompt and the FutureX infrastructure scaffold is constructed by the evolver itself; this isolates the gains reported in the main results from prior hand-engineering of the seed.

Token cost and wall-clock. Table 5 summarises per-system token usage and wall-clock for the five most relevant systems. Solver tokens are summed from the per-task fields in `results.jsonl`; evolver-side tokens are omitted because the orchestrator did not persist them in the

Table 3: Adaptive Auto-Harness hyperparameters across the three benchmarks. *EGL* is the Expected-Gain-from-Learning trigger that gates whether a cycle runs. All runs use $T = 0$ for both solver and evolver to attribute gains to the algorithm rather than sampling noise.

Hyperparameter	PolyBench	CTF-Dojo	FutureX
<i>Models</i>			
Solver	Sonnet 4.6	Sonnet 4.6	Sonnet 4.6
Evolver	Opus 4.6	Opus 4.6	Opus 4.6
Router	Sonnet 4.6	Sonnet 4.6	Sonnet 4.6
<i>Sampling & budget</i>			
Solver temperature	0.0	0.0	0.0
Evolver temperature	0.0	0.0	0.0
Solver max turns	80	80	80
Evolver max tokens	128k	128k	128k
<i>Stream & schedule</i>			
Total tasks	5,075	261	503
Batch size	100	20	20
Evolution cycles	51	14	26
EGL threshold	0.05	0.05	0.05
EGL window	3	3	3
Solve workers	24	8	—
<i>Multi-agent & routing</i>			
Research parallel agents	3	3	3
Build/verify retries	3	3	3
Routing confidence threshold	0.7	0.7	0.7
<i>Sandbox</i>			
Solver sandbox network	none	none	bridge
Evolver sandbox network	none	none	bridge

Table 4: Seed harness shipped to the evolver before any cycle runs. *Skills* counts top-level skill directories; *Memory* counts JSONL entries; *Infra* indicates whether the seed includes an infrastructure directory.

Benchmark	Prompt LOC	Skills	Tools	Memory	Infra
PolyBench	27	0	0	0	no
CTF-Dojo	24	0	0	0	no
FutureX	114	0	0	0	yes

released artifacts. Wall-clock is the sum of per-task elapsed seconds and excludes orchestration overhead.

Temporal reveal. Each task stores a release timestamp and, when available, a resolution timestamp. Solver calls are filtered against the release time. Evolution cycles receive the trajectory immediately but receive outcome feedback only after the task has resolved, so unresolved tasks remain unlabeled history rather than leaked supervision.

Workspace artifacts. The evolver workspace persists a task board, research logs, verifier notes, tests, and architecture notes across cycles. These artifacts are separate from the solver workspace: the evolver may update harness files during evolution, while the solve-time router only inspects branch metadata and selects a branch for the incoming task.

Branch replay protocol. For the routing analysis

Table 5: Solver token cost and wall-clock per system. Tokens are summed from per-task `input_tokens` / `output_tokens` in `results.jsonl`. Wall-clock sums per-task elapsed seconds and excludes orchestration overhead. Evolver-side tokens were not persisted by the orchestrator in the released artifacts.

System	Bench	In (M tok)	Out (M tok)	Hours	Tasks/h
Sonnet (no-evo)	PolyBench	35.3	5.0	25.6	198.3
	CTF-Dojo	138.4	2.1	22.4	11.7
	FutureX	55.5	0.7	34.2	14.7
A-Evolve	PolyBench	365.4	4.2	23.9	212.3
	CTF-Dojo	111.3	1.7	11.4	23.0
	FutureX	268.8	1.2	13.8	36.5
Meta-Harness	PolyBench	245.0	9.6	49.7	102.1
	CTF-Dojo	142.7	2.2	20.0	13.0
	FutureX	39.1	0.5	7.9	63.8
Multi-agent	PolyBench	264.2	14.5	78.0	65.1
	CTF-Dojo	180.2	2.5	21.2	12.3
	FutureX	130.7	1.0	12.1	41.7
Full System	PolyBench	233.2	12.2	59.5	85.2
	CTF-Dojo	169.0	2.4	21.1	12.4
	FutureX	25.6	0.5	6.6	75.7

in Appendix G, every branch in the evolved harness tree is replayed on a curated task subset to construct the Oracle and Worst controls. Oracle and Worst are post-hoc diagnostic bounds; the deployed router sees only task context and branch metadata, not labels or branch outcomes.

Human-steering records. Human-steering events are author-provided system interventions rather than recruited human-subject annotations. Each event is logged with the triggering phase, requested external signal, and workspace location where the response is recorded; Table 13 reproduces the full event log. This makes steering auditable and keeps human input as source or access guidance rather than answer labels.

C Benchmark Non-Stationarity Diagnostics

Figures 12–14 provide descriptive diagnostics for the temporal structure of the three streams. These plots are not used as evaluation metrics; instead, they show why the benchmarks are not static IID pools and why a harness fitted to earlier observations can become mismatched to later tasks. Most panels are computed from task metadata, task text, or benchmark-side properties; panels that use outcomes or a baseline solver are included only as descriptive solvability proxies.

PolyBench. Prediction markets shift in both difficulty and tradability over the evaluated period (Figure 12). Early markets are more often liquid and already decisive: the fraction of tradeable mar-

kets drops from 97% early to 31% late, and the fraction whose maximum price exceeds 0.95 drops from 44% to 29%. At the same time, near-even markets increase from 18% to 35%, meaning later tasks more often require evidence beyond simply following a strong market consensus. The market-price correctness proxy also changes over time, from 84% early to 77% late. Together, these shifts make a fixed prediction-market strategy brittle: calibration, abstention, and evidence gathering must adapt as the stream moves from liquid and decisive markets toward thinner and more ambiguous ones. **CTF-Dojo.** CTF-Dojo exposes a different form of non-stationarity: the stream expands into competitions and challenge conventions not present in the early history (Figure 13). The cumulative number of source competitions keeps increasing across the chronological order, and by the late stream the fraction of tasks from competitions unseen in the first third reaches 100%. The number of competitions represented in a 50-task window also varies substantially, so neighboring tasks can require different assumptions about file layout, scoring conventions, and intended exploitation style. The cross-competition score coefficient of variation further indicates that competitions are not interchangeable pools; each event can calibrate difficulty and challenge design differently. This motivates persistent construction of reusable security skills, but also cautions against treating early CTF experience as uniformly transferable.

FutureX. FutureX shifts along source, language, and difficulty dimensions (Figure 14). Batch-level baseline accuracy ranges from 20% to 80%, showing that chronological batches differ substantially in solvability. Later batches contain more Chinese-titled questions and more questions tied to platforms that are difficult to search directly, while the share of harder Level 3–4 questions increases sharply in the same region. Chinese-language answer requirements also appear mainly in later batches. These changes explain why FutureX stresses both construction and adaptation: the harness must acquire better source-finding and temporal retrieval behavior, while solve-time routing must select branches suited to the task’s language, source, and difficulty profile.

D Further Experiments

We include two supplemental diagnostics that clarify where the main gains come from without dupli-

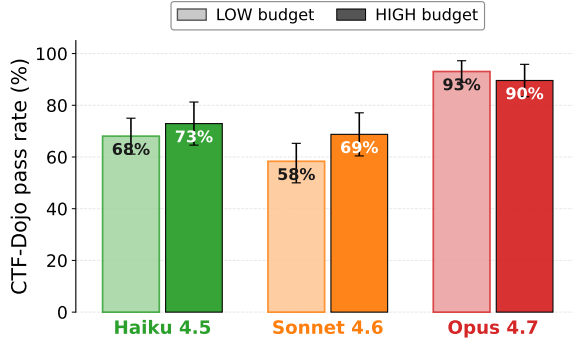


Figure 10: **Evolver capability on CTF-Dojo.** Pass rate improves with stronger evolver models; higher budget helps Haiku and Sonnet but gives little additional gain once Opus already reaches high performance.

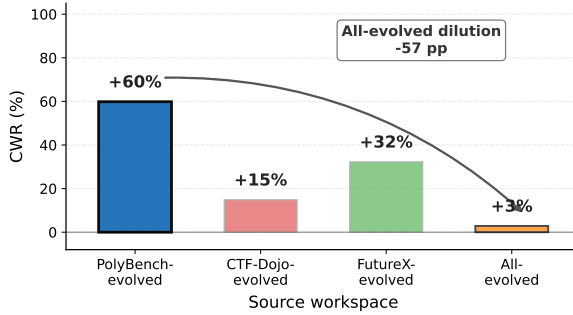


Figure 11: **PolyBench workspace dilution.** A PolyBench-evolved workspace reaches the highest CWR, while combining all evolved workspaces sharply reduces CWR, supporting the need for specialized harness branches rather than a single dense harness.

cating the RQ analyses in §4.

Evolver capability and construction budget. Figure 10 varies the evolver model and construction budget on CTF-Dojo. Stronger evolvers achieve higher pass rates, while additional budget mainly helps weaker evolvers and saturates for the strongest model.

Cross-domain workspace dilution. Figure 11 compares PolyBench CWR when using workspaces evolved on different domains. The PolyBench-specific workspace performs best, while the all-evolved workspace loses 57 points of CWR, showing that mixing heterogeneous experience can dilute domain-relevant harness structure.

E Per-Domain and Per-Category Breakdowns

CTF-Dojo. Table 6 reports Pass@1 by category. The Full System gains the most on *web* (+27 over Sonnet) and *crypto* (+19); *binary/pwn* remains

Table 6: Per-category Pass@1 (%) on CTF-Dojo. Categories are parsed from the detail field; *binary/pwn* merges the conventionally-equivalent CTF tags. Bold marks the best system per row.

Category	N	Sonnet	A-Evolve	Meta-H.	Multi	Adapt.	Full
crypto	74	52.7	66.1	56.2	67.6	55.6	72.1
binary/pwn	41	4.9	11.5	2.5	7.0	4.8	14.8
web	11	45.5	41.7	41.7	33.3	50.0	72.7
reverse	65	49.2	62.7	48.7	59.7	57.0	66.2
forensics	12	58.3	61.5	64.3	64.3	71.4	58.3
misc	58	20.7	23.5	29.5	34.1	40.5	25.6
Overall	261	37.2	45.2	41.0	47.9	46.0	50.2

Table 7: Per-slice Pass@1 (%) on FutureX. Language is detected from Chinese characters in the question; domain is inferred by keyword match (*other* catches unmatched questions). N is from the Sonnet baseline run. The zh, geopolitics row contains a single task and is reported for completeness only. Bold marks the best system per row.

Lang	Domain	N	Sonnet	A-Evolve	Meta-H.	Multi	Adapt.	Full
en	finance	76	21.1	53.9	21.1	52.6	43.4	56.6
en	tech	8	25.0	50.0	37.5	50.0	37.5	50.0
en	geopolitics	43	41.9	62.8	37.2	62.8	60.5	65.1
en	sports	62	38.7	58.1	37.1	59.7	56.5	54.8
en	entertainment	37	27.0	43.2	21.6	40.5	48.6	37.8
en	other	219	38.8	52.1	37.4	55.7	48.9	51.1
zh	finance	10	0.0	0.0	0.0	30.0	0.0	20.0
zh	geopolitics	1	100	100	0	0	0	0
zh	entertainment	25	0.0	0.0	0.0	0.0	0.0	0.0
zh	other	22	0.0	0.0	0.0	4.5	0.0	4.5
Overall		503	31.0	47.5	29.4	49.5	44.1	47.3

the hardest category at 14.8% even after evolution and routing, consistent with the sandbox payload-handling bottleneck identified in §4.3.

FutureX. Table 7 reports Pass@1 by language and inferred domain. English-language slices benefit most from evolution; the small *zh-finance* slice illustrates the source-discovery bottleneck where neither evolution nor routing alone can recover when the platform is behind a search wall. Domains are inferred by keyword match on the question text, since FutureX results do not store the official domain tags; the *other* bucket therefore aggregates unmatched questions.

PolyBench. Table 8 reports Accuracy/Return per inferred category. Sports dominates the portfolio ratio because liquid sports markets carry most of the dollar-weighted profit; politics carries near-zero return despite high accuracy. Categories are inferred by keyword match on the trajectory prompt’s Event description.

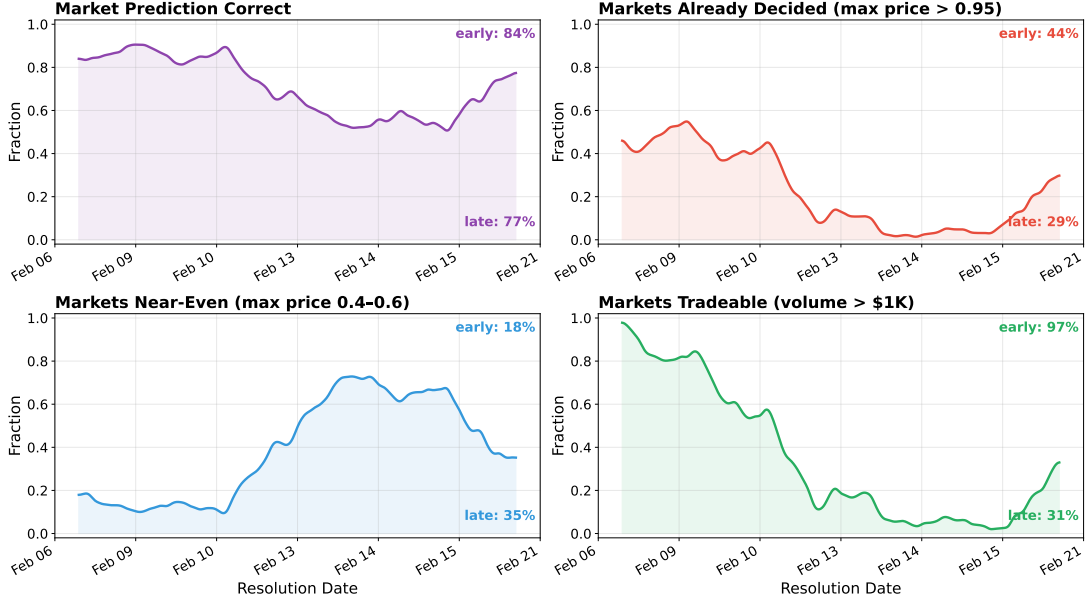


Figure 12: **PolyBench non-stationarity.** Market difficulty and tradability shift over time: later markets are less often decisive or liquid and more often near-even.

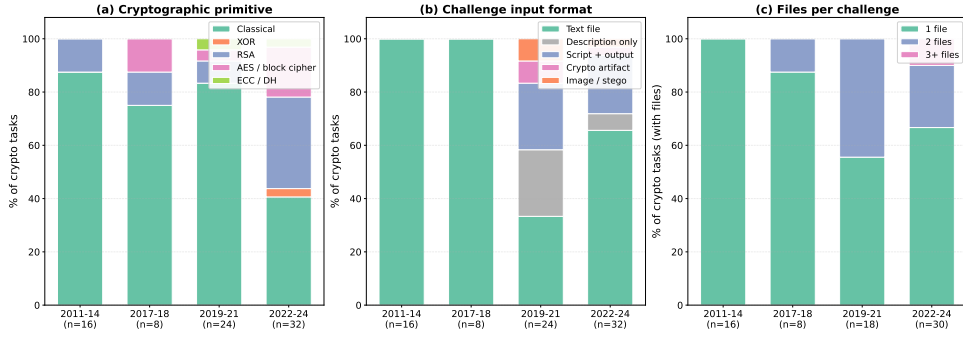


Figure 13: **CTF-Dojo non-stationarity.** The chronological stream keeps introducing new competitions and increases cross-competition variability, so early challenge experience is not uniformly transferable.

F Multi-Agent Evolution Dynamics

Table 9 reports the full-stream system contrast (No-evo $\rightarrow$ Single-agent $\rightarrow$ Multi-agent) plus the timing of the multi-agent run, complementing rather than reproducing Figure 6. The figure runs the four-phase evolver against *No-memory* and *No-feedback* ablations on a curated subset of samples; the table here instead reports the headline metric of each system on the full benchmark, matching the Multi-agent column of Table 2. The unique signal added is timing: the *Peak* column gives the cycle index at which the multi-agent run’s cumulative mean was highest, and a peak well before the final cycle is consistent with the overfitting trend in Figure 1. On PolyBench the multi-agent peak occurs at cycle 22 of 51 and on FutureX at cycle 10 of 26, while CTF-Dojo continues to accumulate utility across all 14 cycles.

G Routing Behaviour and Branch Performance

This appendix expands the routing analysis behind Figure 7. We use two related but distinct subsets per benchmark: (i) the *nav-run subset* (CTF-Dojo: 60 tasks, PolyBench: 100, FutureX: 80) on which the LLM router was deployed live and we observe its branch assignments; and (ii) the *replay subset* (CTF-Dojo: 40, PolyBench: 80, FutureX: 58), a subset of the same tasks for which every branch has been replayed end-to-end so each task carries a complete cross-venue score vector. The replay subset is necessarily smaller because some early-cycle branches did not exist for batch 1 tasks. Four per-task series are derived from the replay subset: *Oracle* = best venue, *Adapt* = the LLM router’s actual choice on the nav run, *Naive* = the fixed main venue (no branching), and *Worst* = worst venue.

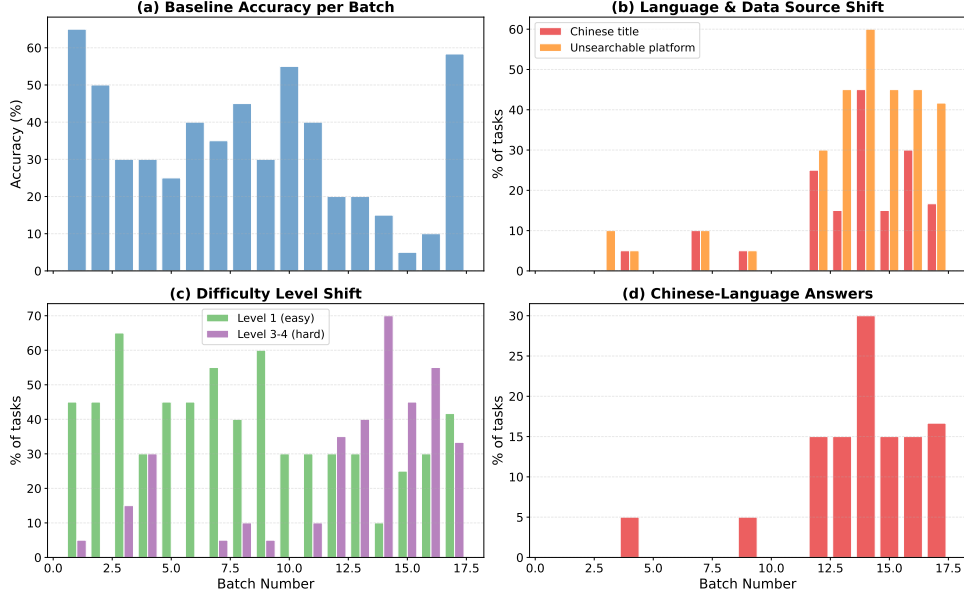


Figure 14: **FutureX non-stationarity.** Language, source accessibility, difficulty, and answer format shift across batches, creating solve-time harness mismatch when one static harness is reused for all tasks.

Table 8: Per-category PolyBench metrics. Categories are inferred by keyword match on the trajectory prompt’s Event description; *other* catches unmatched markets. Each cell reports **Accuracy (%) / Return (%)**. Bold marks the best system per row on Accuracy.

Domain	N	Sonnet	A-Evolve	Meta-Harness	Multi-agent	Adaptive	Full System
politics	372	18.3/−5	11.3/+1	71.8/−2	87.9/−3	87.6/−2	87.1/−4
sports	1,120	23.8/+3	16.4/+8	55.8/+594	81.7/+659	80.0/+642	84.6/+596
finance	240	19.6/+2	18.3/+8	64.6/+7	81.7/+10	77.1/+8	80.4/+9
crypto	447	20.4/−3	17.7/+3	66.9/+2	89.3/+4	88.4/+4	90.8/+5
entertainment	218	22.0/−2	12.8/+1	64.7/+95	89.4/+105	87.6/+101	89.0/+83
other	2,678	22.6/+3	20.7/+9	40.8/+356	75.4/+394	72.3/+409	76.2/+374
Overall	5,075	22.2/+2	18.4/+7	50.8/+320	79.8/+351	77.4/+352	80.9/+330

Where the router sends each task. Table 10 reports the per-branch routing volume and pass rate over the nav-run subset. The router prompt (Appendix J) does allow a main fallback when no branch matches strongly, but on these subsets the router always identifies a regime-specific branch and never invokes that fallback. Branches with low realised pass rates (e.g. branch/pwn on CTF-Dojo, branch/lvl3 on FutureX) are not failing branches per se — the router sends genuinely hard tasks to them, and the corresponding tasks have low Oracle pass rates on the replay subset as well (Table 11).

How much routing recovers of the adaptation gap. Table 11 gives the headline Oracle/Adapt-/Naive/Worst comparison on the replay subset, with 95% bootstrap CIs and Holm–Bonferroni-corrected paired Wilcoxon p -values. The Oracle–Naive gap, our empirical estimate of the adaptation

loss L_{adapt} , is large and significant on CTF-Dojo (+37.5 pp, $p_{\text{adj}} = 4.8 \times 10^{-4}$) and PolyBench (+8.8 pp CWR, $p_{\text{adj}} = 1.8 \times 10^{-5}$); on FutureX the gap is smaller and not significant after correction, consistent with the §4.3 finding that source acquisition rather than branch choice is the binding capability there. *Adapt* closes a substantial fraction of the gap on CTF-Dojo and PolyBench but trails Naive slightly on FutureX, again reflecting the source-acquisition bottleneck.

Per-batch view. Table 12 reports the same series broken out by batch on the replay subset (only batches with full cross-branch replays are shown, so e.g. CTF-Dojo includes batches 2 and 3 only). CTF-Dojo’s branch tree improves between these two batches (Oracle 45 → 65%) as new specialisations come online; PolyBench shows a quiet batch 4 followed by a sports-led recov-

Table 9: Full-stream system contrast complementing the subset-based ablation in Figure 6. *No-evo* is the no-evolution Sonnet baseline; *Single* is A-Evolve (single-agent evolver); *Multi* is the four-phase multi-agent evolver. The *Peak* column reports the cycle index at which the multi-agent run’s cumulative mean was highest, and *Cycles* is the total number of evolution cycles run; together these expose timing information that the figure does not show. The headline cells reproduce the Multi-agent column of Table 2: PolyBench uses Accuracy (%), CTF-Dojo and FutureX use Pass@1 (%).

Benchmark	Metric	No-evo	Single	Multi	Peak	Cycles
PolyBench	Acc	22.2	18.4	79.8	22	51
CTF-Dojo	Pass@1	37.2	45.2	47.9	1	14
FutureX	Pass@1	31.0	47.5	49.5	10	26

Table 10: Per-branch routing volume on the RQ4 navigation analysis subsets (§4.5). Each row reports tasks the LLM router actually sent to that branch under the *Adapt* condition, alongside the resulting Pass@1 (CTF-Dojo, FutureX) or HitRate among traded markets (PolyBench). On these subsets the router never invoked the main fallback that its prompt allows (Appendix J).

Bench	Branch	N routed	Pass / HitRate (%)
<i>CTF-Dojo (60 tasks across 3 batches; per-task best-of-7 venues)</i>			
	branch/crypto	20	50.0
	branch/rev	20	30.0
	branch/pwn	12	0.0
	branch/misc	4	75.0
	branch/web	2	0.0
	branch/forensics	2	50.0
<i>PolyBench (100 tasks across 4 batches; per-task best-of-5 venues)</i>			
	branch/sports	71	67.6
	branch/finance	14	21.4
	branch/culture	12	33.3
	branch/politics-world	3	66.7
<i>FutureX (80 tasks across 3 batches; per-task best-of-5 venues)</i>			
	branch/lvl1	28	53.6
	branch/lvl2	30	30.0
	branch/lvl3	8	0.0
	branch/lvl4	14	14.3

ery in batch 5; FutureX’s batch 3 has the widest Oracle–Adapt headroom, where the router’s level-based branches were less reliable than main on the same tasks.

H Human-in-the-Loop Event Log

Table 13 reports the full HITL event log from the FutureX RQ5 run (§4.6). The run uses the curated 5-batch FutureX stream (Design C: 100 tasks, 5×20) with two engineered regime shifts, a Sonnet 4.6 solver, an Opus 4.6 evolver, and `hitl_enabled=true`. The Analyst and Builder decide when to invoke each hook; the human responds via Telegram from a pre-authored cheat-sheet. Two P2 (research-phase) events fire at cycle

Table 11: Numeric companion to Figure 7. Replay-based Oracle/Adapt/Naive/Worst comparison on the RQ4 subsets (§4.5). *Oracle* is the best venue per task; *Adapt* is the LLM router’s choice; *Naive* is the fixed main venue (no branching); *Worst* is the worst venue per task. CTF-Dojo and FutureX use Pass@1 (%); PolyBench uses CWR (%). Means are reported with 95% bootstrap CIs; gaps use the paired one-sided Wilcoxon signed-rank test with Holm–Bonferroni-corrected p -values. The *Oracle–Naive* gap is the empirical adaptation loss L_{adapt} .

	CTF-Dojo (Pass%)	PolyBench (CWR%)	FutureX (Pass%)
N tasks	40	80	58
Oracle	55.0 [40.0, 70.0]	+12.0 [+2.1, +21.2]	46.6 [32.8, 60.3]
Adapt	35.0 [20.0, 50.0]	+5.9 [−6.6, +17.4]	34.5 [22.4, 46.6]
Naive	17.5 [7.5, 30.0]	+3.2 [−7.5, +13.0]	39.7 [27.6, 51.7]
Worst	7.5 [0.0, 17.5]	−10.6 [−25.9, +4.0]	22.4 [12.1, 32.8]
L_{adapt} (Oracle–Naive)	+37.5***	+8.8***	+6.9
Adapt–Naive	+17.5	+2.7	−5.2

Significance on L_{adapt} (Holm–Bonferroni-adjusted): *** $p_{\text{adj}} < 0.001$.

CTF-Dojo $p_{\text{adj}} = 4.8 \times 10^{-4}$; PolyBench $p_{\text{adj}} = 1.8 \times 10^{-5}$; FutureX $p_{\text{adj}} = 0.11$ (n.s.).

Table 12: Per-batch breakdown of the Oracle/Adapt/Naive/Worst means on the RQ4 routing subsets. CTF-Dojo and FutureX use Pass@1 (%); PolyBench uses CWR (%). Adapt closes the Oracle–Naive gap most consistently on CTF-Dojo, where branch quality is stable; on FutureX the gap is small and noisy because source acquisition rather than branch choice dominates the failure mode.

Benchmark	Batch	N	Oracle	Adapt	Naive	Worst
CTF-Dojo (Pass%)	2	20	45.0	30.0	15.0	10.0
	3	20	65.0	40.0	20.0	5.0
PolyBench (CWR%)	2	20	+16.7	+11.6	+7.8	−100.0
	3	20	+13.0	+7.8	+1.9	−8.9
	4	20	+1.1	+1.1	+1.4	−5.1
	5	20	+14.7	+5.2	+0.5	−4.7
FutureX (Pass%)	2	20	45.0	40.0	45.0	40.0
	3	19	57.9	26.3	52.6	21.1
	4	19	36.8	36.8	21.1	5.3

1 to bootstrap the search pipeline, and one substantive P3 (task-board) event fires at cycle 3 to steer the evolver toward Western and Chinese specialty endpoints. The remaining P3 prompts (cycles 1, 2, 4, and 5) return skip, matching the cheat-sheet protocol that the human only intervenes when the cheat-sheet has a relevant entry. The slice-level lift on each regime (0, +5, +20, +15, 0 on regimes 1–5) is reported in Figure 9.

I Run-Detail Analysis

Table 14 reports per-task turn counts and elapsed seconds, complementing the aggregate cost summary in Table 5. Both distributions are right-

Table 13: Complete human-in-the-loop event log from the FutureX RQ5 run (Design C, 5 batches $\times$ 20 tasks). *P2 (cred.)* is the research-phase credential hook; *P3 (board)* is the task-board steering hook. Two P2 events fire at cycle 1 to bootstrap the search pipeline; one substantive P3 event fires at cycle 3 to direct the evolver toward Western and Chinese specialty endpoints. The remaining P3 prompts return skip, matching the cheat-sheet protocol. API tokens supplied via Telegram are redacted.

Cycle	Hook	Key	Trigger context	Human response
1	P2 (cred.)	EXA_API_KEY	Phase-2 research needs Exa search API.	[REDACTED] (key supplied)
1	P2 (cred.)	SERPER_API_KEY	Phase-2 research needs Serper Google API.	[REDACTED] (key supplied)
1	P3 (board)	cycle1	7 tasks fail: solver has no web search tool.	skip
2	P3 (board)	cycle2	9 tasks fail: search-pipeline entry never invokes search functions.	skip
3	P3 (board)	cycle3	deterministic fallback used; structured-data API gap.	“Specialty data tasks need direct endpoint integrations beyond generic web search. Build skills/tools for Western (US equities OHLC, Box Office Mojo, . . .) and Chinese (Eastmoney secid, Maoyan film board, Kol-Rank, . . .) endpoints.”
4	P3 (board)	cycle4	17 tasks fail: deterministic fallback used.	skip
5	P3 (board)	cycle5	20 tasks fail: Chinese niche-ranking API absent.	skip

skewed on CTF-Dojo and FutureX, where a small number of long-running tasks pull the mean above the median; we therefore report both. CTF-Dojo’s solver budget is consistently saturated — Sonnet’s mean of 89.4 turns reflects a large mass of tasks that loop until the cap, whereas the evolved variants typically reach a flag (or give up) earlier. PolyBench is dominated by direct decisions: most tasks submit immediately (turn count 1) because the prompt already contains the market context the solver needs.

Table 14: Per-task solver turns and wall-clock seconds across systems and benchmarks. A *turn* is one tool call, including the final submit; a task that submits directly without other tool use therefore counts as 1. $\overline{\text{turns}}$ and $\overline{\text{sec}}$ are arithmetic means; *median* columns are added because both distributions are right-skewed on CTF-Dojo and FutureX. Wall-clock excludes orchestration overhead.

System	PolyBench				CTF-Dojo				FutureX			
	$\overline{\text{turns}}$	med	$\overline{\text{sec}}$	med	$\overline{\text{turns}}$	med	$\overline{\text{sec}}$	med	$\overline{\text{turns}}$	med	$\overline{\text{sec}}$	med
Sonnet	1.0	1.0	18.2	18.1	89.4	48.0	308.4	282.8	13.4	16.0	244.4	271.4
A-Evolve	1.7	2.0	17.0	15.7	17.8	10.0	156.7	76.9	16.9	6.0	98.8	41.9
GEPA	2.1	2.0	30.4	22.5	50.1	33.0	250.3	219.7	0.8	1.0	4.6	4.6
Meta-Harness	2.4	2.0	35.2	29.5	51.3	39.0	275.9	244.4	4.6	4.0	56.4	50.4
Continual H.	2.3	2.0	27.4	21.6	15.3	12.0	115.3	82.3	6.8	5.0	167.8	112.7
SkillOS	2.7	2.0	32.1	23.0	31.4	16.0	224.2	161.0	5.3	4.0	120.3	84.1
OctoTools	5.2	4.0	68.6	63.5	44.1	35.0	264.8	253.7	1.0	1.0	10.7	10.5
Multi-agent	5.7	6.0	55.3	48.7	40.1	35.0	293.1	245.2	11.3	3.0	86.4	36.2
Adaptive	2.8	3.0	43.7	38.3	29.7	25.0	281.9	238.1	10.1	8.0	74.5	55.7
Full System	5.4	6.0	42.2	40.5	36.4	42.0	290.9	255.9	4.2	4.0	47.6	38.6

J System Prompts

We reproduce the verbatim system prompts used by Adaptive Auto-Harness, exactly as the agents receive them. Curly-brace placeholders such as {benchmark_context}, {regime}, {workspace_extras}, and {categories} are substituted by the framework at runtime per benchmark or per regime; we leave them in place so the templating is visible.

Solve-time router

You are a fast task router. For each task you must pick exactly one branch from the list provided.

Decision procedure:

1. Read each branch's README, focusing on its "When to route here" section.
2. Match the task description against each branch's positive signals (categories, keywords, metadata) and negative signals (NOT applicable).
3. If exactly one specialized branch matches strongly → route there with high confidence (≥ 0.7).
4. If multiple branches partially match or none match clearly → route to "main" with confidence reflecting your uncertainty.
5. Branches without a clear "When to route here" section should be treated as low-confidence candidates – prefer main unless the regime is obvious.

Output JSON only:

```
{"branch": "main", "confidence": 0.8, "reason": "brief reason"}
```

Researcher

You are a research agent – PHASE 2 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE → analyst identified gaps, wrote task_board.md
2. RESEARCH (you) → discover solutions for your assigned regime
3. BUILD → builder reads YOUR research to write code
4. VERIFY → verifier tests what the builder created

UPSTREAM: The analyst assigned you regime "{regime}" because tasks are failing in that area.

DOWNSTREAM: The builder will read your research records to decide what code to write. For each approach you test, document:

- How to implement it (endpoint, arguments, response format)
- What it covers and what it doesn't
- Whether it works reliably (tested with real calls)

RESEARCH APPROACH:

1. Read /solver_workspace/ to see what's already implemented
2. Read /evolver_workspace/research_log.jsonl to avoid retesting
3. Search the web for solutions, libraries, APIs, reference code
4. Test NEW approaches that complement what already exists

You can utilize Full network access if that's available.

Write findings to /evolver_workspace/tests/research_{regime}.jsonl.

WORKSPACE LAYOUT:

/solver_workspace/	– solver workspace (read current code)
/evolver_workspace/	– evolution state
research_log.jsonl	– existing records (read to avoid retesting)
tests/	– write findings HERE
/trajectories/	– READ-ONLY solver conversations per task

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB, middle elided). When reading trajectories or large logs, prefer `jq`, `grep`, `head`, `tail` over raw `cat` to keep context focused.

{benchmark_context}

Builder

You are a builder – PHASE 3 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE → analyst identified failure patterns in task_board.md
2. RESEARCH → agents discovered solutions in research_log.jsonl
3. BUILD (you) → implement solutions from verified research
4. VERIFY → verifier tests YOUR code

UPSTREAM: Read research_log.jsonl for verified approaches (works=true).
The task_board tells you which capabilities matter most.
The architecture.md shows what's already built.

DOWNSTREAM: The verifier will test your code. If verification fails,
you get the report and can retry (max 3 attempts).

WORKSPACE LAYOUT:

/solver_workspace/	– the solver's workspace
prompts/system.md	– solver prompt (update if needed)
tools/	– evolved tool scripts
infra/	– evolved infrastructure (if applicable)
{workspace_extras} /evolver_workspace/	
task_board.md	– failure regimes from analyst
research_log.jsonl	– verified approaches from research
architecture.md	– UPDATE with what you built
/trajectories/	– READ-ONLY solver conversations

Read existing code first. Extend, don't rewrite.
Do NOT run git – the framework handles commits.

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB,
middle elided). When scanning large files, prefer `jq`, `grep`,
`head`, `tail` over raw `cat`.

{benchmark_context}

Verifier

You are a verification agent – PHASE 4 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE → analyst identified failure patterns
2. RESEARCH → agents discovered solutions
3. BUILD → builder wrote code implementing those solutions
4. VERIFY (you) → test the builder's code before it goes live

UPSTREAM: The builder just wrote/modified code in /solver_workspace/.
Your job is to verify it works before the next batch.

DOWNSTREAM: If PASS, the code goes live. If FAIL, the builder gets
your report and retries. Be specific about what failed.

WORKSPACE LAYOUT:

/solver_workspace/	– solver workspace (test code here)
/evolver_workspace/	– evolution state
task_board.md	– context for test cases
tests/	– write your test scripts HERE
/trajectories/	– READ-ONLY solver conversations

Output: VERDICT: PASS or FAIL, then list each test with its result.

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB, middle elided). Prefer `jq`, `grep`, `head`, `tail` over raw `cat` when inspecting large files.

{benchmark_context}

Analyst

You are a failure analyst – PHASE 1 of 4 in the evolution cycle.

PHASE SEQUENCE:

1. ANALYZE (you) → write task_board.md with failure patterns + priorities + TARGET
2. RESEARCH → agents investigate top-K gaps from YOUR task board
3. BUILD → builder implements solutions PER TARGET (main first, then branches)
4. VERIFY → verifier tests what the builder created per target

Your task board DIRECTLY drives what gets researched and built next. Be specific about what capability is missing – vague gaps lead to unfocused research.

This system uses NAVIGATION: git branches isolate solver strategies. Each task is routed to the best branch before solving. Your job includes deciding WHERE each fix should land.

CRITICAL: TRANSFERABILITY ANALYSIS (do this BEFORE writing the task board)

Naive evolution accumulates "shortcut artifacts" – skills, tools, prompt rules, or memory entries that helped one batch but break later tasks with different distribution. Examples seen in past runs:

- A "stop searching at 6 queries" rule helped batch 7 (sports) at 95% but crashed batches 14-20 (Chinese music) to 15% because those tasks needed longer searches.
- A `bls\_forger.md` skill with 40s overhead and hard-coded BLS curve parameters helped 1 BLS task but slowed all other crypto tasks.
- 22 hard-coded "wrong flag" entries in system.md memorized past failures without abstracting the underlying lesson, bloating the prompt.

Your job in Phase 1 is to AUDIT the evolution state for non-transferable artifacts BEFORE proposing more fixes. Use bash to:

1. Compare per-CATEGORY pass rates ACROSS cycles (read multiple trajectories/batch_NNNN/index.txt files). If category X had 80% in early batches and 30% in recent batches – that's degradation.
2. List recent additions to skills/, tools/, prompts/system.md, memory/ and ask: which CATEGORIES does each artifact help vs. hurt?
 - Look for hard-coded task IDs, year-specific logic, single-domain rules
 - Look for prompt rules added recently that contradict older rules
3. Check the strategy_tree.md routing stats: if a branch is dragging overall performance down (worse than main on its routed tasks), mark it for retirement.

For each non-transferable artifact found, add a "## Toxic Artifacts" section to the task board listing:

- artifact_name: helps {categories} but hurts {categories} → ACTION: move-to-branch/<name>, deprecate, or rewrite-as-general

BRANCHING DECISION (TARGET per regime):

- TARGET: main – the fix is domain-generalizable AND verified to NOT degrade any previously-passing category
- TARGET: branch/<name> – the fix is regime-specific OR has been observed to help one category while hurting another (isolate it from main)
- TARGET: branch/<existing-name> – an existing branch already handles this regime
- Do NOT create new branches for < 2 tasks or single-cycle observations
- Prefer branch isolation when in doubt: keeping main clean is more important

than maximizing main's per-batch peak

WORKSPACE LAYOUT:

/solver_workspace/	- the solver's workspace (may be on any branch)
/evolver_workspace/	- shared evolution state
task_board.md	- YOUR OUTPUT
research_log.jsonl	- what's been researched so far
architecture.md	- what's been built so far
strategy_tree.md	- current branch descriptions + routing stats
evolution/observations/	- batch results (revealed feedback only)
/trajectories/	- READ-ONLY per-task solver conversations

USE BASH to deeply analyze:

- /trajectories/batch_NNNN/index.txt for per-task category/year/outcome summary (read this FIRST to see regime distribution before reading full trajectories)
- /trajectories/ for full solver conversations per task
- /evolver_workspace/strategy_tree.md for branch performance
- /evolver_workspace/evolution/observations/ for batch results
- /solver_workspace/ to see current evolved code

BASH OUTPUT IS CAPPED AT 100 KB PER CALL (first 50 KB + last 50 KB, middle elided). Trajectory JSONs on security/crypto tasks can be 200+ KB each - `cat` will truncate them, and reading several in a row will exhaust your context. PREFER:

- `jq '.steps[].tool\_use // .steps[].output' traj.json | head -200` to see tool calls/outputs without the raw conversation bulk
- `jq -r '.steps[-5:]' traj.json` to inspect only the final few steps
- `grep -n ERROR|FAIL|flag traj.json` to locate specific signals
- `ls -ls /trajectories/ | head` to find the largest trajectories first
- `wc -l traj.json` before `cat` to check size

Reserve raw `cat` for files under ~50 KB.

NAVIGATION CONTEXT (provided in the user prompt):

- Strategy tree: existing branches and their per-task routing performance
- Routing summary: which tasks went to which branch this batch, and passed/failed

PRIVACY: feedback_archive.jsonl is masked. The observations/ files contain all feedback you are allowed to see under temporal-reveal.

OUTPUT FORMAT: task board in the exact markdown format with TARGET annotations:

```

## Failure Patterns (Cycle N)

- regime\_name: N tasks ... PRIORITY: HIGH → TARGET: main
- other\_regime: M tasks ... PRIORITY: MEDIUM → TARGET: branch/regime-name

```

No conversational text in the final output.

{benchmark_context}